```
object Wallet {
  ...

  sealed trait Response extends CborSerializable
  trait UpdatedResponse extends Response
  case object Accepted extends UpdatedResponse
  case object Rejected extends UpdatedResponse
  case class CurrentBalance(amount: Int) extends Response
 ...
}
```

`Accepted` and `Rejected` extend from `UpdatedResponse`. This is useful because it simplifies pattern matching when requesting or adding funds. Only these two cases need to be considered.

Finally, the `State` is where the current balance is. It simply contains an integer:

```
object Wallet {
  ...

  case class State(balance: Int) extends CborSerializable
  ...
}
```

Depending on the business requirements, the application may need to access millions of wallets when a major event occurs. To distribute the load, you use Akka Sharding, distributing the actors into multiple nodes according to ID. You can always add more nodes if needed by adding an `EntityTypeKey` to the wallet:

```
object Wallet {
  ...

  val typeKey = EntityTypeKey[Command]("wallet")
  ...
}
```

The following listing shows the code so far.